

```
36.
37.     # Check for a trade
38.     if curr['Signal'] == 1 and prev['Signal'] == 0:
39.         # Calculate how many shares to buy
40.         available_capital = prev['Cash'] * position_size
41.         price = curr['Adj Open'] # Using next day's open price
42.         max_shares = np.floor(available_capital / price)
43.
44.         # Calculate trading costs
45.         commission = max_shares * commission_per_share
46.         slippage = max_shares * price * (slippage_percent / 100)
47.         total_cost = (max_shares * price) + commission + slippage
48.
49.         # Execute the trade if we have enough capital
50.         if total_cost <= prev['Cash']:
51.             backtest.loc[backtest.index[i], 'Trade'] = max_shares
52.             backtest.loc[backtest.index[i], 'Position'] = max_shares
53.             backtest.loc[backtest.index[i], 'Cash'] = prev['Cash'] - total_cost
54.             backtest.loc[backtest.index[i], 'Trade Cost'] = commission + slippage
55.
56.     elif curr['Signal'] == 0 and prev['Signal'] == 1:
57.         # Sell all shares
58.         price = curr['Adj Open']
59.         shares_to_sell = prev['Position']
60.
61.         # Calculate trading costs
62.         commission = shares_to_sell * commission_per_share
63.         slippage = shares_to_sell * price * (slippage_percent / 100)
64.
65.         # Execute the sale
66.         sale_value = (shares_to_sell * price) - commission - slippage
67.         backtest.loc[backtest.index[i], 'Trade'] = -shares_to_sell
68.         backtest.loc[backtest.index[i], 'Position'] = 0
69.         backtest.loc[backtest.index[i], 'Cash'] = prev['Cash'] + sale_value
70.         backtest.loc[backtest.index[i], 'Trade Cost'] = commission + slippage
```